

Тема: «Язык сценариев JavaScript. Переменные»

Содержание:

- 1 Основные соглашения языка JavaScript
 - 1.1 Правила записи идентификаторов
 - 1.2 Зарезервированные слова
- 2 Включение сценариев JavaScript в HTML-документ
 - 2.1 Контейнер `<script>...</script>`
 - 2.2 Параметры тега `<script>`
 - 2.3 Meta-тег и сценарии
- 3 Переменные языка сценариев JavaScript
 - 3.1 Время жизни переменных
 - 3.2 Создание переменных
 - 3.3 Строковые переменные
 - 3.4 Числовые переменные
 - 3.5 Булевы переменные
 - 3.6 Переменные-массивы
 - 3.7 Конструктор `Object()`
- 4 Преобразование строк и чисел

1 Основные соглашения языка JavaScript

Как и любой язык программирования JavaScript использует константы, переменные для хранения данных определенного типа, операторы – обозначения для выполнения определенных действий, литералы – буквальное обозначения для переменных, выражения – комбинации переменных, литералов и операторов, задающие способ вычисления некоторого значения. Для обозначения всех этих объектов используются идентификаторы. Идентификатор – это уникальное имя, которое присваивается какому-либо объекту.

Основные соглашения синтаксиса JavaScript:

- язык JavaScript чувствителен к регистру
- приложение JavaScript представляет собой набор отдельных инструкций, которые могут располагаться в отдельной строке каждая, так и в одной строке несколько инструкций. Инструкции отделяются друг от друга `;`. Если инструкции располагаются по одной строке, то `;` можно опускать. Если одна инструкция разделяется на несколько строк, то никакие символы продолжения инструкции не требуются. Исключение составляют текстовые литералы (текст, представляющий собой значение некоторой переменной) – текстовый литерал нельзя разделять на две или несколько строк;

Программные блоки (тело функции или часть кода, которая выполняется в цикле или является последовательностью действий, определяемой выражением условного оператора) записывается в фигурных скобках `{...}`.

1.1 Правила записи идентификаторов

Идентификаторы могут начинаться с любой буквы (от `a` до `z` или `A-Z`) либо с символа подчеркивания `_`, оставшаяся часть может содержать цифры, символы подчеркивания и буквы.

В именах различаются символы верхнего и нижнего регистра.

Например, `MyVariable` – это не то, что `myvariable`.

1.2 Зарезервированные слова

Это идентификаторы, которые не могут использоваться в качестве имен переменных, функций, объектов и методов.

abstract	false	this	do	public
boolean	final	throw	float	return
break	finally	throws	for	short
byte	instanceof	transient	function	static
case	int	true	goto	try
catch	interface	class	if	typeof
char	longNative	const	implements	var
else	new	continue	import	void
enum	null	debugger	package	volatile
export	switch	default	private	while
extends	synchronized	delete	protected	with

2 Включение сценариев JavaScript в HTML-документ

2.1 Контейнер `<script>...</script>`

Включение сценариев JavaScript в HTML-документ осуществляется с помощью контейнера `<script>...</script>`, который в принципе может располагаться в любом месте документа. Документ может содержать неограниченное количество таких контейнеров как в разделе `<head>` так и в разделе `<body>`. Внутри контейнера `<script>` операторы JavaScript дополнительно помещаются в контейнер комментариев HTML для того, чтобы предотвратить выдачу содержимого контейнера `<script>` на экран в качестве обычного содержимого теми браузерами, которые не поддерживают тег `<script>`. Таким образом, структура для внедрения сценария имеет следующий вид:

```
<script language="JavaScript" type="text/javascript">
<!-- скрыть сценарий от браузеров, не поддерживающих JavaScript
операторы JavaScript
операторы JavaScript
//-->
</script>
```

Следует обратить внимание на то, что перед символом закрытия контейнера комментариев добавлено `//` – это метка начала комментариев в языке JavaScript. Наличие этой метки позволяет интерпретатору JavaScript правильно обработать сценарий, при отсутствии метки `//` интерпретатор считает строку `-->` строкой сценария и выдаст сообщение об ошибке, так как оператора `-->` в языке нет. Впрочем, для самых последних версий браузеров это уже не требуется.

2.2 Параметры тега `<script>`

Параметры тега `<script>`:

`<script`

`defer` – уведомляет браузер, что сценарий не генерирует никакого содержимого, выводимого в HTML-документ, то есть метод `document.write(...)` не используется, наличие данного параметра в этом случае ускоряет загрузку страницы;

`language="JavaScript"` – это отмененный, но на практике широко использующийся параметр, сообщающий на каком языке (и версии языка) следует сценарий, причем если браузер не поддерживает данную версию языка, то весь сценарий будет проигнорирован;

`src=url` – относительная или абсолютная ссылка на имя внешнего файла, содержащего код сценария. В случае языка JavaScript тип файла *.js – в остальных это обычный текстовый файл.

`type="text/javascript"` – формальный параметр для задания языка.
`"text/vbscript"`

2.3 Meta-тег и сценарии

В HTML-документе, содержащем сценарии рекомендуется также в разделе `<head>` вставлять следующую инструкцию:

```
<meta http-equiv="content-script-type" content="text/javascript">
```

Наличие этой инструкции задает язык сценариев для данного документа, использующийся по умолчанию. Это ускоряет выполнение сценариев, так как не требует от браузера дополнительного анализа заголовка url-запроса, в котором также имеется подобная информация и которая извлекается оттуда при отсутствии данной инструкции. Кроме того, наличие данной инструкции не мешает в конкретном контейнере `<scripttype="text/vbscript">` использовать другой язык сценариев.

Для обработки ситуации, когда браузер не поддерживает сценарии, используется контейнер `<noscript>...</noscript>`, в котором можно разместить текст и инструкции для этого случая.

3 Переменные языка сценариев JavaScript

Переменные языка JavaScript могут хранить значения различных типов:

- 1) **строки (string)** – последовательность символов;
- 2) числовые значения – **целые (number)**, **действительные числа (number)**;
- 3) **булевы значения (boolean)** – только два значения true или false;
- 4) **null** — специальное значение. Оно имеет смысл «ничего».

```
var age = null;
```

Код выше говорит о том, что возраст age неизвестен.

- 5) **undefined** — специальное значение, которое, как и null, образует свой собственный тип. Оно имеет смысл «значение не присвоено».

Если переменная объявлена, но в неё ничего не записано, то ее значение как раз и есть “undefined”:

```
var u;
alert(u); //выведет "undefined"
```

Можно присвоить undefined и в явном виде, хотя это делается редко:

```
var x=123;
x=undefined;
```

В явном виде undefined обычно не присваивают, так как это противоречит его смыслу. Для записи в переменную «пустого значения» используется null.

- 6) Объекты **Object**.

Первые 5 типов называют «примитивными».

Особняком стоит шестой тип: «объекты». К нему относятся, например, даты, он используется для коллекций данных и для многого другого. Позже мы вернёмся к этому типу и рассмотрим его принципиальные отличия от примитивов.

В языке JavaScript тип данных может с легкостью переопределяться.

3.1 Время жизни переменных

Время жизни переменной связано с окном, в котором они созданы и зависит от того, где они определены. JavaScript-программы содержатся в документах, и при запуске нового документа в браузер любые переменные, созданные в программе, будут удалены.

Чтобы сохранить какие-либо значения при загрузке нового документа в JavaScript имеются только два решения, а именно:

- путем создания переменных во фреймосодержащем документе верхнего уровня;
- путем использования “cookies”.

Для сохранения переменных и функций с помощью фреймосодержащих документов эти переменные и функции необходимо определить в документе верхнего уровня, а затем использовать свойства parent или top объекта window для обращения к ним из документов, загруженных в фреймы. Следует обратить внимание а то, что такие переменные и функции следует задавать в заголовке документа верхнего уровня между тегами <head>...</head>. Конечно, существует вероятность, что документ может быть загружен без фрейма и тогда обращение к неопределенному объекту вызовет ошибку, но можно предварительно проверить загружен ли документ в фрейм:

```
<script >
if(self.parent.frames.length==0){
document.writeln("Эта страница загружена в браузер без фрейма");
}
</script>
```

Cookies позволяет сохранять небольшие фрагменты информации на локальном диске пользователя. Устанавливая значения cookies проверяя их на последующих стадиях работы, можно восстанавливать необходимые значения.

3.2 Создание переменных

В языке JavaScript создать переменную гораздо проще, чем в других языках программировании. Например, при создании переменной нет необходимости указывать ее тип. Переменные определяют как с начальными значениями, так и без них. В процессе выполнения программы уже созданные переменные можно даже приводить к различным типам данных.

Переменную можно создавать одним из способов:

- при помощи оператора var и операции присавивания =;
- при помощи операции присваивания.

Основное различие между этими способами состоит в том, что с помощью оператора var обязательно должны создаваться временные переменные, то есть только те, которые будут использоваться внутри функций или циклов. Глобальные переменные создаются как с помощью var, так и при помощи оператора присваивания.

Оператор var используют не только для создания переменной, но и для ее инициализации. Операция присваивания (=) необходима, чтобы запомнить в переменной значение. При работе с переменными, созданными без начального значения, ее использовать не обязательно.

Например, операция присваивания

```
varMyVariable = 35
```

создает переменную с именем MyVariable, содержащую числовое значение 35. Переменная существует до тех пор, пока загружен текущий документ. Если эта переменная создается во

фреймосодержащем документе верхнего уровня, к ней следует обращаться с помощью выражения `top.MyVariable`, а еще лучше `parent.MyVariable`, для подстраховки от вложенных фреймов.

В языке JavaScript переменные можно переопределять, даже задавая другой тип данных. Например, после выполнения оператора

```
varMyVariable = "35"
```

переменная будет уже хранить строку "35". Подобные преобразования приводят иногда к недоразумениям. Например, скрипт-программа, представленная ниже.

```
<!DOCTYPEHTMLPUBLIC "-//W3C//DTDHTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title>Примеры преобразования переменных</title>
<meta http-equiv="content-type" content="text/html; charset=windows-1251">
<meta http-equiv="content-script-type" content="text/javascript">
</head>

<body>

<script language="JavaScript" type="text/javascript">
<!-- скрыть сценарий от браузеров, не поддерживающих JavaScript
var a=35,b="5",c;
c=a+b;
c=c-a;
document.writeln("C="+c);
//-->
</script>
</body>
</html>
```

Какое значение примет переменная "с" после выполнения программы? Значение переменной "с" после завершения блока будет равным числовому значению 320. О принципах преобразования типов переменных будет рассказано позже.

Переменную можно определить и не используя оператор "var", а просто достаточно присвоить значение, причем каков тип данных будет присвоен, такого типа и окажется переменная. Оператор "var" используется по большей части для читабельности JS-программы. Переменная может быть задана и без начальных значений, например:

```
var MyVariable;
```

Создана переменная с именем MyVariable, не имеющая определенного типа данных и начального значения. Переменные, создаваемые при помощи таких описаний, известны как неопределенные (**undefined**). Например, сравнивая такую переменную со значением null, можно узнать определена ли переменная. Однако нельзя путать разные вещи: "" – пустая строка – это строковый тип и вовсе не null-значение.

Тип переменной может быть установлен в любом месте JS-программы в отличие от других языков программирования, что дает дополнительную гибкость, но и возможность запутаться. К переменной, которая не создана, обратиться не возможно. Если нужно создать временную переменную, например, счетчик цикла, нужно записать перед ней префикс var:

```
for (var i=0; i<= document.links.length; i++) {
...}
```

В данном примере инициализируется переменная i и затем используется в качестве счетчика оператора for.

Переменные, определенные с помощью var, разделяются запятой. Для большей читабельности JavaScript-программы каждую переменную записывают с новой строки.

3.3 Строковые переменные

Строка представляет собой множество символов, заключенных в одинарные или двойные кавычки. Строки в JavaScript рассматриваются как объекты. Это практически самый распространенный вид объектов, поэтому в JavaScript имеется множество стандартных функций, предназначенных для управления строками. Создать объект String можно одним из нескольких способов.

- присваивание значения при помощи конструктора String();
- использование оператора var и оператора присваивания для создания и инициализации строки;
- создание и инициализация строковой переменной в операторе присваивания;
- преобразование переменной числового типа путем сложения со строковым типом (10+""=>"10").

Самым простым и наиболее распространенным способом создания объекта String() является использование таких операторов, как

```
var myVariable="Текстовая строка";
```

Приведенный оператор присваивает строку "Текстовая строка" строковой переменной myVariable. Переменная myVariable рассматривается как строковый объект и может использовать любой из стандартных методов объекта String языка JavaScript. Оператор var можно пропустить, как и говорилось ранее, он нужен в основном для читабельности программы.

Для создания строковых объектов допускается использовать конструктор String() с оператором new. Рассмотрим пример:

```
var myVariable = new String();
```

Этот оператор создает новый объект – пустую строку с именем myVariable. Изначально эта пустая строка (""), а значение свойства myVariable.length равно 0.

Конструктор String() допускает передачу заданной строки в виде аргумента, например:

```
var myVariable=new String("Текстовая строка");
```

Строковый объект может содержать **специальные символы, управляющие форматированием строк**:

- \n – символ новой строки;
- \r – символ возврата каретки;
- \f – код перехода на новую страницу;
- \xnn – представление символа в виде шестнадцатичного ASCII-кода nn;
- \b – код клавиши [Backspace].

Эти символы будут правильно интерпретированы только в контейнерах <textarea> и в методах "Alert".

Форматирование же для document.write() осуществляется другими методами или тегами HTML.

Объект String имеет свойство length, значением которого является количество символов в строке, содержащейся в объекте. Методы объекта String можно разделить на две категории:

- методы форматирования HTML-документа;
- методы обработки строк.

Методы форматирования документа применяются для вывода строк в документ, а методы управления строками – для проверки и изменения содержимого объекта String.

3.4 Числовые переменные

Числовые значения могут быть либо целыми, либо числами с плавающей точкой. Числа с плавающей точкой называют действительными или вещественными. К числовым значениям применимы следующие операции:

- умножение (*);
- деление (/);
- сложение (+);
- вычитание (-);
- увеличение (++);
- уменьшение (--);

Кроме того, используют операции умножения, деления, сложения и вычитания в сочетании с присваиванием (*=, /=, +=, -=), а также методы объекта Math.

3.5 Булевы переменные

Булевы или логические переменные содержат только литеральные значения true и false, используются в логических выражениях и операторах.

Для проверки значения булевой переменной используют и операцию логического равенства: `Boolean var == true`, хотя в данном случае операция проверки излишняя. Для проверки значений, которые не являются истинными, используют знак логического отрицания (!). Например, выражение `!boolean var` возвратит значение true, если значение `boolean var` равно false. Вместо слов “true” и “false” можно использовать числовые значения 1 и 0, поскольку именно так булевы значения представлены в памяти компьютера.

3.6 Переменные массивы

Переменные массивы содержат упорядоченные наборы значений одного типа, для удобства представленные в виде одной переменной. Многие стандартные объекты-свойства документа в языке JavaScript, в частности гиперсвязи и метки, являются массивами. В языке JavaScript элементу массива следует обращаться при помощи выражения

arrayName[index]

где `arrayName` – имя массива, а `index` – числовая переменная или число, задающее позицию элемента в массиве. Например, `arrayName[0]` является первым элементом этого массива. Индексы элементов массива в языке JavaScript начинаются с нуля. Элементы массива могут быть любого типа, например, строками или булевыми переменными. Кроме того, при определенных условиях массив может содержать элементы различных типов данных. В языке JavaScript массивы создаются при помощи

- конструктора `Array()`;
- конструктора `Object()`;
- конструктора, определенного пользователем.

С помощью конструктора `Array()` не только создают объект `array`, но и присваивают начальные значения его элементам. Существует возможность добавлять элементы в массив динамически – путем присваивания определенных значений элементам массива. Допускается также «пропускать» элементы массива и задавать их в любой последовательности. Для создания нового экземпляра объекта `array` конструктор `Array()`

необходимо использовать с оператором `new`. Например, в следующем примере создается массив с именем `arrayImg`, содержащий два элемента, каждый из которых является объектом `String`.

```
var path = "c:/images";
arrayImg = new Array();
arrayImg[0] = path + "img1.gif";
arrayImg[1] = path + "img2.gif";
```

При использовании конструктора `Array()` значение свойства `length` возвращает значение 2. Элементы массива также могут быть заданы как параметры конструктора:

```
var path = "c:/images";
arrayImg = new Array(path + "img1.gif", path + "img2.gif");
```

Данное выражение представляет собой сокращенную запись предыдущего примера.

В языке Javascript можно создавать массив, в котором элементы имеют различный тип данных:

```
var myArray = new Array(3.14, true, 85, date(), "word");
```

создает массив, элемент `myArray[0]` которого является числом с плавающей запятой, элемент `myArray[1]` – булевым значением, элемент `myArray[3]` – объектом `Date`.

Размер массива и, следовательно, значение свойства `length` объекта, создаваемого конструктором `Array()`, зависят от максимального значения индекса, который применялся для задания элемента массива. Например:

```
var myArray = new Array;
myArray[20] = "Это 21 элемент массива";
```

двадцать первому элементу массива присваивается строковое значение “Это 21 элемент массива”, а значение свойства `myArray.length` равно 21 независимо от того имеют ли значения элементы массива с индексом меньше 20.

Значение свойства `length` объекта `Array` автоматически устанавливается при явном указании количества элементов в конструкторе `Array()`:

```
myArray = new Array(10);
```

оператор создаст массив из 10 элементов от 0 до 9. Значение свойства `length` массива нельзя установить путем присваивания, так как `length` является свойством только для чтения. Например, чтобы задать значение 10 для свойства `length`, нужно только определить значение последнего, в данном случае 9-го элемента массива.

```
myArray = new Array();
myArray[9] = 0;
```

Кроме того, существует возможность задать значения элементов массива при его конструировании. Например:

```
myArray = newArray(0,0,0,0,0,0);
```


3.7 Конструктор Object()

Конструктор Object() и Array() работают по-разному. Конструктору Object() невозможно передать несколько элементов массива, поэтому такое выражение

```
var myObj = Object(value1, value2);
```

работать не будет. Массивы, создаваемые при помощи конструктора Object(), не имеют свойства length. Поэтому, создавая массив таким способом, следует либо организовывать обход этого массива посредством цикла for и подсчитать элементы массива, либо жестко задать длину массива как значение его первого элемента (обычно так поступают моделируя свойство length). Затем обращаться к нему по мере необходимости для проверки размера массива, увеличения значения при добавлении нового элемента, а также в качестве параметра цикла при циклическом считывании или изменении значений элементов. Такой объект часто бывает неприемлем для случаев, когда содержимое массива должно динамически изменяться, поэтому в большинстве случаев пользуются конструктором Array(). Значения индексов массивов, создаваемых в конструкторе Object(), также начинаются с нуля. Для создания массива при помощи конструктора Object() принято использовать запись вида:

```
var myObj = new Object();
myObj[0] = 2; //задаем размерность массива
myObj[1] = "Первый элемент";
myObj[2] = "Второй элемент";
```

Чтобы узнать размер массива, созданного подобным образом, необходимо обратиться к элементу myObj[0]. Значением свойства myObj.length является null, так как это значение не определено.

3.8 Преобразование строк и чисел

Преобразование строк и чисел осуществляется одним из следующих способов:

преобразование числа в строку символов производится путем сложения числового аргумента со строковым независимо от перестановки слагаемых. Например, если переменная var1=123, то преобразовать переменную и, следовательно, ее значение в строку символов можно так: var1=var1+"" или наоборот: var1="" + var1. Если сложить не с пустой строкой: var1=var1+"456", то результатом значения переменной var1 станет "123456". Это же справедливо и наоборот: var1="456"+var1 – результат : "456123";

преобразование строки в число производится путем вычитания одного операнда из другого и также независимо от их позиции. Например, если переменная var1="123", то преобразовать ее в число можно, если вычесть из нее значение 0: var1=var1-0, и соответственно, значение переменной из строкового типа преобразуется в числовой: 123. При перестановке операндов соответственно знак числового значения поменяется на противоположный. В отличие от преобразования числа в строку в действиях вычитания нельзя применять буквенные значения. Так если "JavaScript"+10 превратиться в var1 == "JavaScript10", то операция типа var1 = "JavaScript"-10 выдаст значение "NaN", то есть такая операция не допустима. При вычитании строкового значения из строкового также происходит преобразование: var1 = "20"-15, значением переменной var1 станет число 5.